Copy Circuit

개요

Copy Circuit은 EVM 실행 중 발생하는 다양한 데이터 복사 연산의 정확성을 검증하는 회로입니다. CALLDATACOPY, CODECOPY, RETURNDATACOPY, LOG, MCOPY 등의 opcode 실행 시 소스에서 목적지로 바이트 단위 데이터 복사가 올바르게 수행되었는지 검증합니다.

주요 역할:

- 소스와 목적지 간 바이트 단위 복사 검증
- 다양한 데이터 타입 간 복사 지원 (Memory, Bytecode, TxCalldata, TxLog, AccessList)
- 읽기-쓰기 쌍(read-write pair)의 일관성 검증
- 메모리 워드 정렬 및 마스킹 처리
- RW 카운터 관리 및 검증
- 패딩 및 범위 초과 읽기 처리

파일 위치:

- 메인 파일: `zkevm-circuits/src/copy_circuit.rs`
- Gadget 구현: `zkevm-circuits/src/copy_circuit/copy_gadgets.rs`
- 유틸리티: `zkevm-circuits/src/copy_circuit/util.rs`
- 테이블 정의: `zkevm-circuits/src/table.rs`

Copy 데이터 타입

Copy Circuit은 다양한 소스와 목적지 간의 복사를 지원합니다.

CopyDataType 열거형

```
pub  enum CopyDataType {
Memory = 0, // 메모리 영역
Bytecode = 1, // 바이트코드
TxCalldata = 2, // 트랜잭션 CallData
TxLog = 3, // 트랜잭션 로그
RlcAcc = 4, // RLC 누적값 (특수 용도)
Padding = 5, // 패딩 (비활성 행)
AccessListAddresses = 6, // Access List 주소
AccessListStorageKeys = 7, // Access List 스토리지 키
}
```

복사 시나리오 예시

1. **CALLDATACOPY**: TxCalldata → Memory
2. **CODECOPY**: Bytecode → Memory
3. **RETURNDATACOPY**: Memory → Memory
4. **EXTCODECOPY**: Bytecode → Memory
5. **LOG**: Memory → TxLog
6. **MCOPY**: Memory → Memory (같은 call_id 내)
7. **Access List**: AccessListAddresses/StorageKeys → RW Table

Copy Table 구조

Copy Table은 복사 이벤트의 메타데이터를 저장하고 다른 회로와 공유합니다.

Copy Table 스키마

```
pub struct CopyTable {
pub q_enable: Column<Fixed>, // 행 활성화 여부
pub is_first: Column<Advice>, // 이벤트의 첫 번째 행
pub id: Column<Advice>, // 소스/목적지 ID (RLC)
pub addr: Column<Advice>, // 현재 주소
pub src_addr_end: Column<Advice>, // 소스 버퍼 끝 주소
pub real_bytes_left: Column<Advice>, // 남은 바이트 수
pub rlc_acc: Column<Advice>, // RLC 누적값
pub rw_counter: Column<Advice>, // RW 카운터
pub rwc_inc_left: Column<Advice>, // 남은 RW 증가량
pub tag: BinaryNumberConfig<CopyDataType, N_BITS>, // 데이터 타입
}
```

필드 설명

- **is_first**: 복사 이벤트의 첫 번째 read-write 쌍을 표시
- **id**:
- Memory: Call ID 또는 Caller ID
- Bytecode: 바이트코드 해시의 RLC 인코딩
- TxCalldata/TxLog: Transaction ID

- AccessList: Transaction ID
- **addr**: 현재 복사 중인 바이트의 주소 (메모리 주소, 바이트코드 인덱스 등)
- **src_addr_end**: 소스 버퍼의 끝 주소. 이 값 이상의 주소에서 읽으면 0으로 패딩
- **real_bytes_left**: 마스크되지 않은 실제 복사할 바이트 수
- **rlc_acc**: 복사된 데이터의 RLC 누적값 (특정 용도)
- **rw_counter**: 현재 RW 연산의 카운터 값
- **rwc_inc_left**: 이벤트 종료까지 남은 RW 증가량
- **tag**: 현재 행의 데이터 타입

Copy Circuit 구조

주요 컬럼

```
pub struct CopyCircuitConfig<F> {
pub q_step: Selector, // 읽기 행 표시
pub is_last: Column<Advice>, // 마지막 read-write 쌍
pub value: Column<Advice>, // 복사된 값
pub value_prev: Column<Advice>, // 쓰기 전 값
pub value_word_rlc: Column<Advice>, // 워드 RLC (쓰기 후)
pub value_word_rlc_prev: Column<Advice>, // 워드 RLC (쓰기 전)
pub word_index: Column<Advice>, // 워드 내 바이트 인덱스 [0..31]
pub mask: Column<Advice>, // 마스크 여부
pub front_mask: Column<Advice>, // 앞쪽 마스크 여부
pub value_acc: Column<Advice>, // 값 RLC 누적
```

```
pub is_pad: Column<Advice>, // 패딩 여부
// 데이터 타입 플래그
pub is_tx_calldata: Column<Advice>,
pub is_bytecode: Column<Advice>,
pub is_memory: Column<Advice>,
pub is_memory_copy: Column<Advice>, // MCOPY 여부
pub is_tx_log: Column<Advice>,
pub is_access_list_address: Column<Advice>,
pub is_access_list_storage_key: Column<Advice>,
// IsEqual 칩
pub is_id_unchange: IsEqualConfig<F>, // ID 변경 감지
pub is_src_end: IsEqualConfig<F>, // 소스 끝 감지
pub is_word_end: IsEqualConfig<F>, // 워드 끝 감지
pub non_pad_non_mask: Column<Advice>, // !is_pad && !mask
// 외부 테이블
pub tx_table: TxTable,
pub rw_table: RwTable,
pub bytecode_table: BytecodeTable,
}
```

복사 프로세스

Read-Write 쌍 구조

Copy Circuit은 각 바이트 복사를 **읽기 행(reader row)**과 **쓰기 행(writer row)**의 쌍으로 처리합니다.

```
행 구조:

Row 0: Reader (q_step = 1) - 소스에서 읽기

Row 1: Writer (q_step = 0) - 목적지에 쓰기

Row 2: Reader (q_step = 1) - 다음 바이트 읽기

Row 3: Writer (q_step = 0) - 다음 바이트 쓰기

...
```

행 간 연결

- **CURRENT (Rotation(0))**: 현재 행
- **NEXT_ROW (Rotation(1))**: 다음 행 (reader → writer 또는 writer → reader)
- **NEXT_STEP (Rotation(2))**: 다음 단계 (reader → reader 또는 writer → writer)

복사 이벤트 흐름

1. **초기화** (`is_first = 1`):
- 이벤트 파라미터 설정 (id, tag, src_addr_end)
- 초기 주소, 바이트 수, RW 카운터 설정
- value_acc, word_rlc 초기화
2. **바이트 단위 복사** (각 read-write 쌍):
- Reader: 소스에서 값 읽기
- Writer: 목적지에 값 쓰기
- 주소 증가 (마스크 제외)

- bytes_left 감소
- RLC 누적 업데이트
- RW 카운터 업데이트

3. **종료** (`is_last = 1`):

- bytes_left = 0 검증
- rwc_inc_left = 0 검증
- Reader와 Writer의 value_acc 일치 검증

패딩 및 마스킹

소스 패딩 (is_pad)

소스 주소가 `src_addr_end` 에 도달하거나 초과하면 0으로 패딩됩니다.

```
// is_pad 조건
is_pad = (addr >= src_addr_end)


// 패딩 시 값은 0
if  is_pad {
value = 0
}
```

제약 조건:

- `is_first` 일 때: `is_pad = (addr == src_addr_end)`
- 이후: `is_pad_next = (addr_next == src_addr_end) ? 1 : is_pad`

- Writer 행에서는 항상 `is_pad = 0`

마스킹 (mask)

메모리 워드 정렬을 위해 일부 바이트는 마스크 처리됩니다.

```
// 마스크 구조
mask = front_mask + back_mask


// front_mask: 첫 워드의 앞부분 (복사 시작 전)
// back_mask: 마지막 워드의 뒷부분 (복사 종료 후)
```

제약 조건:

- `mask`, `front_mask`, `back_mask` 는 모두 boolean
- `front_mask` 는 감소만 가능 (1 → 0)
- `back_mask` 는 증가만 가능 (0 → 1)
- `front_mask` 와 `back_mask` 는 동시에 1일 수 없음
- 마스크된 바이트: `value = value_prev` (쓰기 없음)
- LOG, Access List는 front_mask 없음

non_pad_non_mask

실제 복사된 데이터를 식별하기 위한 플래그:

```
non_pad_non_mask = !is_pad && !mask
```

이 플래그는 룩업 조건을 단순화하고 degree를 줄이는 데 사용됩니다.

워드 정렬 및 RLC

워드 인덱스 (word_index)

메모리와 TxLog는 32바이트 워드 단위로 처리됩니다.

```
// word_index: 워드 내 바이트 위치 [0..31]
// 초기값: 0
// 업데이트: (word_index + 1) % 32

// 워드 끝 감지
is_word_end = (word_index == 31)
```

워드 RLC (value_word_rlc)

각 워드의 RLC를 계산하여 메모리 룩업에 사용합니다.

```
// 초기화
word_rlc = value[0]

// 누적
word_rlc_next = if  is_word_end {
0  // 다음 워드 시작
} else {
word_rlc * r + value_next
```

```
}
```

쓰기 전후 RLC:

- `value_word_rlc` : 쓰기 후 워드 RLC
- `value_word_rlc_prev` : 쓰기 전 워드 RLC

값 RLC 누적 (value_acc)

마스크되지 않은 실제 복사 데이터의 RLC를 계산합니다.

```
// 초기화
value_acc = if  non_pad_non_mask { value } else { 0 }


// 누적 (패딩 시 0, 마스크 시 복사)
value_acc_next = if  mask_next {
value_acc  // 마스크: 복사
} else {
value_acc * r + (if  is_pad_next { 0 } else { value_next })
}


// 종료 시 검증
assert(value_acc_reader == value_acc_writer)
```

RW 카운터 관리

RW 연산 타입

RW 테이블 룩업이 필요한 데이터 타입:

- **Memory**: 읽기/쓰기 모두 RW 연산
- **TxLog**: 쓰기만 RW 연산
- **AccessList**: 읽기/쓰기 모두 RW 연산

RW 카운터 업데이트

```
// RW 연산 발생 조건
is_rw_type = is_memory + is_tx_log + is_access_list
is_row_end = if  is_access_list {
!is_reader  // Access List는 writer 행에서만
} else {
is_word_end  // Memory/TxLog는 워드 끝에서
}

rwc_diff = is_rw_type * is_row_end

// rwc_inc_left 감소
rwc_inc_left_next = rwc_inc_left - rwc_diff

// 종료 시 검증
assert(rwc_inc_left_last == 0)
```

```
// rw_counter와 rwc_inc_left의 합은 일정
rw_counter + rwc_inc_left = constant
```

MCOPY 특수 처리

MCOPY (Memory Copy)는 같은 call_id 내에서 메모리를 복사합니다.

```
// MCOPY 감지
is_memory_copy = (src_id == dst_id) &&
(src_type == Memory) &&
(dst_type == Memory)

// MCOPY의 RW 카운터 처리
// - 읽기 단계와 쓰기 단계가 독립적으로 RW 카운터 증가
// - 일반 복사와 달리 read-write 쌍이 아닌 각 단계별로 처리
// - rwc_inc_left[writer] = rwc_inc_left[reader] / 2
```

룩업 테이블 연동

Memory 룩업

워드 단위로 RW 테이블에 룩업합니다.

```
// 조건: is_memory && is_word_end
// 룩업 데이터
```

```
[
1, // q_enable
rw_counter,
!is_reader, // is_write
RwTableTag::Memory,
id, // call_id
addr - 31, // 워드 시작 주소
0, // field_tag
0, // storage_key
value_word_rlc, // value (쓰기 후)
value_word_rlc_prev, // value_prev (쓰기 전)
0, 0  // aux
]
```

TxLog 룩업

워드 단위로 RW 테이블에 룩업합니다.

```
// 조건: is_tx_log && is_word_end
// 룩업 데이터
[
1, // q_enable
rw_counter,
1, // is_write (항상 쓰기)
```

```
RwTableTag::TxLog,
id, // tx_id
addr - 31, // byte_index || field_tag || log_id
0, 0,
value_word_rlc, // value
0, 0, 0
]
```

Bytecode 룩업

바이트 단위로 Bytecode 테이블에 룩업합니다.

```
// 조건: is_bytecode && non_pad_non_mask
// 룩업 데이터
[
1, // q_enable
id, // code_hash (RLC)
BytecodeFieldTag::Byte,
addr, // byte_index
value  // byte value
]
```

TxCalldata 룩업

바이트 단위로 Tx 테이블에 룩업합니다.

```
// 조건: is_tx_calldata && non_pad_non_mask

// 룩업 데이터

[

1, // q_enable

id, // tx_id

TxContextFieldTag::CallData,

addr, // byte_index

value  // byte value

]
```

Access List 룩업

Access List는 주소와 스토리지 키를 별도로 처리합니다.

Access List Address

```
// Tx 테이블 룩업

[

1,

tx_id,

TxContextFieldTag::AccessListAddress,

index, // addr

address  // value_word_rlc (20바이트 주소)
```

```
]


// RW 테이블 룩업
[
1,
rw_counter,
1, // is_write
RwTableTag::TxAccessListAccount,
tx_id,
address, // value_word_rlc
0, 0,
1, // is_warm
is_warm_prev, // value_prev
0, 0
]
```

Access List Storage Key

```
// Tx 테이블 룩업
[
1,
tx_id,
TxContextFieldTag::AccessListStorageKey,
index, // value (인덱스)
```

```
storage_key  // value_word_rlc_prev
]


// RW 테이블 룩업
[
1,
rw_counter,
1, // is_write
RwTableTag::TxAccessListAccountStorage,
tx_id,
address, // value_word_rlc
0,
storage_key, // value_word_rlc_prev
1, // is_warm
is_warm_prev, // value_prev
0, 0
]
```

주요 제약 조건

1. 첫 번째 및 마지막 행 제약

```
// is_first는 reader 행에서만 1
```

```
is_first * !is_reader = 0


// is_last는 writer 행에서만 1

is_last * is_reader = 0


// is_first와 is_last는 boolean

is_first * (1 - is_first) = 0

is_last * (1 - is_last) = 0
```

2. 파라미터 전달

이벤트가 계속되는 동안 파라미터를 다음 단계로 전달합니다.

```
// 조건: !is_last_step

id[0] = id[2]

tag[0] = tag[2]

src_addr_end[0] = src_addr_end[2]
```

3. 종료 보장

모든 이벤트는 반드시 종료되어야 합니다.

```
// 이벤트가 시작되면 (tag != Padding)

// 다음 단계가 활성화되어야 함

if  tag != Padding {
```

```
q_enable[NEXT_STEP] = 1

}


// 결국 is_last = 1이 되어야 tag가 Padding으로 변경 가능
```

4. 주소 증가

```
// 조건: is_continue

// front_mask가 아닌 경우 주소 증가

addr_next = addr + !front_mask
```

5. 바이트 수 감소

```
// 조건: is_continue

// 마스크되지 않은 바이트만 카운트

bytes_left_next = bytes_left - !mask


// 종료 시

if !is_continue {

bytes_left = 0

}
```

6. 워드 완성 검증

RW 타입의 마지막 단계는 워드 끝에서 종료되어야 합니다.

```
// 조건: is_last_step && is_rw_word_type

is_word_end = 1
```

구현 예시

CALLDATACOPY 예시

```
CALLDATACOPY(dest_offset=0, offset=0, length=3)

CallData: [0x12, 0x34, 0x56]
```

Copy Event:

- src_type: TxCalldata
- dst_type: Memory
- src_id: tx_id
- dst_id: call_id
- src_addr: 0
- dst_addr: 0
- src_addr_end: 3
- length: 3

Copy Rows:

```
Row 0 (Reader):
```

```
q_step=1, is_first=1, is_last=0

tag=TxCalldata, id=tx_id, addr=0, value=0x12

is_pad=0, mask=0, front_mask=0, word_index=0

value_acc=0x12, value_word_rlc=0x12



Row 1 (Writer):

q_step=0, is_first=0, is_last=0

tag=Memory, id=call_id, addr=0, value=0x12

value_prev=0x00, value_word_rlc_prev=0x00

Row 2 (Reader):

q_step=1, is_first=0, is_last=0

tag=TxCalldata, id=tx_id, addr=1, value=0x34

word_index=1

value_acc=0x12*r + 0x34, value_word_rlc=0x12*r + 0x34



Row 3 (Writer):

q_step=0, is_first=0, is_last=0

tag=Memory, id=call_id, addr=1, value=0x34



Row 4 (Reader):

q_step=1, is_first=0, is_last=0

tag=TxCalldata, id=tx_id, addr=2, value=0x56

word_index=2
```

```
value_acc=(0x12*r + 0x34)*r + 0x56
```

```
Row 5 (Writer):
q_step=0, is_first=0, is_last=1
tag=Memory, id=call_id, addr=2, value=0x56
bytes_left=0, rwc_inc_left=0
```

MCOPY 예시

```
MCOPY(dest=64, src=0, length=32)
같은 call_id 내에서 메모리 복사
```

특징:

- `is_memory_copy = 1`
- Reader와 Writer가 독립적으로 RW 카운터 증가
- `rwc_inc_left[writer] = rwc_inc_left[reader] / 2`

최적화 기법

1. non_pad_non_mask 컬럼

룩업 조건을 단순화하여 degree를 줄입니다.

```
// 기존: is_bytecode * !is_pad * !mask
// 최적화: is_bytecode * non_pad_non_mask
```

2. IsEqual 칩 재사용

자주 사용되는 비교 연산을 칩으로 구현:

- `is_id_unchange` : ID 변경 감지
- `is_src_end` : 소스 끝 도달 감지
- `is_word_end` : 워드 끝 감지

3. 데이터 타입 플래그

각 데이터 타입별 boolean 컬럼을 사용하여 조건부 제약 조건을 효율적으로 구현합니다.

테스트 전략

Copy Circuit은 다양한 복사 시나리오를 테스트합니다.

단위 테스트

- 각 CopyDataType별 복사
- 패딩 처리 (소스 범위 초과)
- 마스킹 처리 (워드 정렬)
- MCOPY 특수 케이스
- Access List 처리

통합 테스트

- CALLDATACOPY, CODECOPY, RETURNDATACOPY 등 opcode 실행
- 다양한 길이와 오프셋 조합
- 메모리 워드 경계 케이스
- 여러 복사 이벤트 연속 실행

공식 테스트

`testool` 을 통해 이더리움 공식 테스트 실행:

- 복사 관련 opcode 테스트
- Edge case 검증

관련 문서

- EVM Circuit - 복사 opcode 실행
- State Circuit - RW 테이블 검증
- Bytecode Circuit - 바이트코드 데이터 제공
- Tx Circuit - CallData 및 Access List 제공
- RW Table - 메모리 읽기/쓰기
- Copy Table - 복사 이벤트 데이터

참고 자료

- 코드: `zkevm-circuits/src/copy_circuit.rs`
- Gadgets: `zkevm-circuits/src/copy_circuit/copy_gadgets.rs`
- 테이블: `zkevm-circuits/src/table.rs` (CopyTable)
- 테스트: `zkevm-circuits/src/copy_circuit/test.rs`